

Spring Security

필터 체인, 인증, 인증 아키텍처, 인가, HTTP 요청 인가

작성 기준: 원문 페이지를 확인한 뒤 만든 학습용 한국어 번역 요약본입니다. 공식문서 전체 문장을 그대로 복제한 전문 직역본은 아니며, 원문 URL을 함께 남겼습니다.

[1] 전체 구조 설명

- Spring Security의 Servlet 지원은 Filter 기반이다.
- FilterChainProxy와 SecurityFilterChain은 어떤 보안 필터를 어떤 요청에 적용할지 결정한다.
- 인증은 사용자가 누구인지 확인하는 과정이고, 인가는 인증된 사용자가 어떤 요청이나 메서드를 실행할 수 있는지 결정하는 과정이다.
- HTTP 요청 인가는 authorizeHttpRequests 규칙으로 request level 권한 모델을 만든다.

[2] 문서별 직역체 학습 정리

Security Architecture

- Servlet 애플리케이션에서 요청은 FilterChain을 거쳐 Servlet으로 전달된다.
- Spring Security는 이 Filter 구조를 사용해 exploit protection, authentication, authorization 같은 작업을 삽입한다.
- FilterChainProxy는 Spring Security Servlet 지원의 중심 Filter이며, 여러 SecurityFilterChain으로 위임할 수 있다.
- SecurityFilterChain은 현재 요청에 어떤 보안 Filter들을 적용할지 결정한다.
- 보안 필터는 순서가 중요하다. 예를 들어 인증 필터는 인가 필터보다 먼저 실행되어야 한다.

Authentication

- Authentication 문서는 구체적인 로그인 방식보다 먼저 인증 아키텍처의 큰 틀을 설명한다.
- 지원하는 인증 방식에는 username/password, OAuth 2.0 login, SAML 2.0 login, CAS, remember-me, JAAS, pre-authentication, X509 등이 있다.
- 실무에서는 인증 방식별 문서를 보되, 각 방식이 AuthenticationManager와 SecurityContext에 어떻게 연결되는지 같이 이해해야 한다.

Authentication Architecture

- SecurityContextHolder는 현재 인증된 사용자의 세부 정보를 저장하는 위치다.
- SecurityContext는 Authentication 객체를 담고, Authentication은 principal, credentials, authorities를 가진다.
- GrantedAuthority는 role이나 scope처럼 principal에게 부여된 상위 수준 권한이다.
- AuthenticationManager는 인증을 수행하는 API이고, ProviderManager는 가장 흔한 구현체다.
- ProviderManager는 여러 AuthenticationProvider에게 인증 가능 여부를 위임한다.

Authorization / Authorize HTTP Requests

- 인가 규칙은 인증 방식과 독립적으로 애플리케이션 안에서 일관되게 사용할 수 있다.

- request URI와 method에 관한 규칙을 붙이는 것부터 시작하는 것이 일반적이다.
- authorizeHttpRequests는 요청 수준에서 /admin은 특정 권한, 나머지는 인증 필요 같은 모델을 표현한다.
- AuthorizationFilter는 SecurityContextHolder에서 Authentication을 가져오는 Supplier를 만들고, AuthorizationManager에 요청과 함께 전달해 결정을 맡긴다.
- 접근이 거부되면 AccessDeniedException이 발생하고, 허용되면 필터 체인이 계속 진행된다.
- 기본적으로 AuthorizationFilter는 뒤쪽에 있으므로, 애플리케이션 endpoint는 authorizeHttpRequests에 포함되어야 한다.

[3] 핵심 실행 흐름

HTTP 요청 도착 → FilterChainProxy 진입 → 요청에 맞는 SecurityFilterChain 선택 → CSRF/헤더 등 보호 필터 실행 → 인증 필터가 Authentication 생성 또는 확인 → SecurityContextHolder에 인증 정보 저장 → AuthorizationFilter가 권한 규칙 평가 → 허용 시 DispatcherServlet/Controller로 진행

[4] 중요한 포인트 정리

- 인증(Authentication)과 인가(Authorization)를 섞어 이해하면 Security 설정이 흔들린다.
- SecurityContextHolder는 현재 요청의 사용자 정보를 읽는 핵심 위치다.
- permitAll, authenticated, hasRole 같은 규칙은 요청 흐름에서 AuthorizationFilter가 평가한다.

원문 URL

<https://docs.spring.io/spring-security/reference/servlet/architecture.html>

<https://docs.spring.io/spring-security/reference/servlet/authentication/index.html>

<https://docs.spring.io/spring-security/reference/servlet/authentication/architecture.html>

<https://docs.spring.io/spring-security/reference/servlet/authorization/index.html>

<https://docs.spring.io/spring-security/reference/servlet/authorization/authorize-http-requests.html>